1. Analyze two GenAI programs:

Mistral | GLM 5.3 | Thinking
DeepSeek | DeepSeek-V4.1-Flash-1017 | Thinking

2. How you accessed them:

https://chat.mistral.ai/work/ Mistral | GLM 5.3 | Thinking
https://chat.deepseek.com DeepSeek | DeepSeek-V4.1-Flash-1017 | Thinking

3. Prompt used

Use two GenAI's to modify the Assignment 4 code you receive as follows (Note:
You may use either ChatGPT or CoPilot as one of your choices, but not both.
They are essentially the same GenAI.):

- Write the solutions for each puzzle file to a file named solutionN.txt, where N is the puzzle number.
- Each solutionN.txt file should include the following: o If no solutions are found, a single line: "No solution found."
- o If solutions are found, it should include all solutions followed by a
- single line: "Solutions found: n", where n is the number of solutions found.

The solutions should be in the same format as the input files:
puzzle1.txt – puzzle5.txt.

The AI is also given all of the puzzles.txt tiles

Code: For Lab

```
/*
 * Program: EECS 348 Assignment 4 - Sudoku Solver
 * Description: Solves five Sudoku puzzles using object-oriented
 * programming and recursive backtracking.
 *
 * Inputs: puzzle1.txt, puzzle2.txt, puzzle3.txt,
 *         puzzle4.txt, puzzle5.txt
 *
 * Output: Displays each original puzzle, all possible
 * solutions, or "No solution found".
 *
 * Author: Ethan Church
 * Creation Date: October 13, 2026
 * Collaborators: Trey Curtis
 * Sources: ChatGPT, Gemini
 *
 * Improvements:
 * - Added more detailed input error messages.
 * - Improved comments for readability.
 * - Kept the optimized recursive backtracking method.
 */

#include <iostream>
#include <fstream>
```

```
#include <string>

using namespace std;

class Sudoku {
private:
    int grid[9][9]{};
    long long solutions = 0;
    string errorMessage;

    /*
     * Checks whether a number can be placed in a cell.
     * The number cannot already exist in the same row,
     * column, or 3x3 block.
     */
    bool canPlace(int row, int col, int digit) const {

        // Check the row and column.
        for (int i = 0; i < 9; i++) {
            if (grid[row][i] == digit ||
                grid[i][col] == digit) {
                return false;
            }
        }

        // Find the beginning of the 3x3 block.
        int startRow = (row / 3) * 3;
        int startCol = (col / 3) * 3;

        // Check every cell inside the block.
        for (int r = startRow; r < startRow + 3; r++) {
            for (int c = startCol; c < startCol + 3; c++) {
                if (grid[r][c] == digit) {
                    return false;
                }
            }
        }

        return true;
    }

    /*
     * Uses recursive backtracking to solve the puzzle.
     * Chooses the empty cell with the fewest valid numbers.
```

```
 * If a guess does not work, it removes the number
 * and tries another possibility.
 */
void backtrack() {

    int bestRow = -1;
    int bestCol = -1;
    int fewest = 10;

    // Search for the empty cell with the fewest choices.
    for (int row = 0; row < 9; row++) {
        for (int col = 0; col < 9; col++) {

            // Skip cells that already contain a number.
            if (grid[row][col] != 0) {
                continue;
            }

            int choices = 0;

            // Count the valid numbers for this cell.
            for (int digit = 1; digit <= 9; digit++) {
                if (canPlace(row, col, digit)) {
                    choices++;
                }
            }

            // No valid numbers means this path cannot work.
            if (choices == 0) {
                return;
            }

            // Remember the cell with the fewest choices.
            if (choices < fewest) {
                fewest = choices;
                bestRow = row;
                bestCol = col;
            }
        }
    }

    /*
     * If no empty cells remain, the puzzle is solved.
     * Print the solution and continue searching
```

```
         * for any additional solutions.
         */
        if (bestRow == -1) {
            solutions++;

            cout << "Solution " << solutions << ":" << endl;
            print();

            return;
        }

        // Try numbers 1 through 9 in the selected cell.
        for (int digit = 1; digit <= 9; digit++) {

            if (canPlace(bestRow, bestCol, digit)) {

                // Place the number.
                grid[bestRow][bestCol] = digit;

                // Recursively solve the remaining puzzle.
                backtrack();

                // Remove the number to try another possibility.
                grid[bestRow][bestCol] = 0;
            }
        }
    }

public:

    /*
     * Reads a Sudoku puzzle from a text file.
     * Underscores represent empty cells.
     * Displays a specific error if the file is invalid.
     */
    bool load(const string& filename) {

        ifstream input(filename);

        // Check whether the file exists and can be opened.
        if (!input.is_open()) {
            errorMessage = "Could not open file: " + filename;
            return false;
        }
```

```
char cell;

// Read all 81 cells from the file.
for (int row = 0; row < 9; row++) {
    for (int col = 0; col < 9; col++) {

        // Check for missing puzzle data.
        if (!(input >> cell)) {
            errorMessage =
                "Incomplete puzzle: Expected 81 cells.";
            return false;
        }

        // Convert underscores into empty cells.
        if (cell == '_') {
            grid[row][col] = 0;
        }

        // Convert character digits into integers.
        else if (cell >= '1' && cell <= '9') {
            grid[row][col] = cell - '0';
        }

        // Reject invalid characters.
        else {
            errorMessage =
                "Invalid character found in puzzle.";
            return false;
        }
    }
}

// Check whether the file contains extra data.
if (input >> cell) {
    errorMessage =
        "Invalid puzzle: More than 81 cells found.";
    return false;
}

solutions = 0;
errorMessage = "";

return true;
```

```
}

/*
 * Prints the Sudoku grid.
 * Empty cells are displayed as underscores.
 */
void print() const {

    for (int row = 0; row < 9; row++) {
        for (int col = 0; col < 9; col++) {

            if (grid[row][col] == 0) {
                cout << "_";
            }
            else {
                cout << grid[row][col];
            }

            if (col < 8) {
                cout << " ";
            }
        }

        cout << endl;
    }

    cout << endl;
}

/*
 * Checks whether the original puzzle is valid.
 * Filled cells cannot contain duplicate numbers
 * in the same row, column, or 3x3 block.
 */
bool validStartingGrid() const {

    for (int row = 0; row < 9; row++) {
        for (int col = 0; col < 9; col++) {

            int digit = grid[row][col];

            // Empty cells do not need to be checked.
            if (digit == 0) {
                continue;
```

```
        }

        // Check for duplicates in rows and columns.
        for (int i = 0; i < 9; i++) {

            if (i != col && grid[row][i] == digit) {
                return false;
            }

            if (i != row && grid[i][col] == digit) {
                return false;
            }
        }

        // Find the starting position of the 3x3 block.
        int startRow = (row / 3) * 3;
        int startCol = (col / 3) * 3;

        // Check the block for duplicate numbers.
        for (int r = startRow; r < startRow + 3; r++) {
            for (int c = startCol; c < startCol + 3; c++) {

                if ((r != row || c != col) &&
                    grid[r][c] == digit) {
                    return false;
                }
            }
        }
    }

    return true;
}

// Returns the error message if loading fails.
string getError() const {
    return errorMessage;
}

/*
 * Starts the solving process.
 * Checks the original puzzle before using backtracking.
 * Displays all solutions or reports if none exist.
 */
```

```
    void solve() {

        if (!validStartingGrid()) {
            cout << "Invalid starting puzzle: "
                 << "Conflicting numbers found." << endl;
            cout << "No solution found" << endl;
            cout << "Solutions found: 0" << endl;
            return;
        }

        // Start recursive backtracking.
        backtrack();

        // Report if no valid solutions were found.
        if (solutions == 0) {
            cout << "No solution found" << endl;
        }

        // Display the total number of solutions.
        cout << "Solutions found: " << solutions << endl;
    }
};

/*
 * Main program:
 * Reads and solves all five Sudoku puzzle files.
 * Each puzzle uses a new Sudoku object.
 */
int main() {

    for (int number = 1; number <= 5; number++) {

        // Create the filename for the current puzzle.
        string filename =
            "puzzle" + to_string(number) + ".txt";

        // Create a new Sudoku object.
        Sudoku puzzle;

        cout << "========== " << filename
             << " ==========" << endl;

        // Attempt to load the puzzle.
        if (!puzzle.load(filename)) {
```

```
            // Display the specific error message.
            cout << "Error: " << puzzle.getError()
                 << endl << endl;

            // Continue to the next puzzle.
            continue;
        }

        // Display the original puzzle.
        cout << "Original Puzzle:" << endl;
        puzzle.print();

        // Solve the puzzle and display its solutions.
        puzzle.solve();

        cout << endl;
    }

    return 0;
}
```

2. The two GenAI examples include one other difference.
    a. What is the difference?

Mistral / GLM 5.3: Uses a separate writeGrid() function to write solutions to the output file.
DeepSeek V4.1 Flash: Uses a more general print(ostream& out) function that can write to

Otherwise they are the same because I gave it the code and both of them will perform the same because the base of the code is the same.

Analyze the difference in terms of:
    a. Execution time

writeGrid() and print(solutionOut) fundamentally do the same thing under the hood,, so in terms of execution time they are the same

    b. Space complexity

Because a sudoku board is fundamentally a set or rows and colls they all do the same thing which means that they will have the same time complexity and both fundamentally use a recursive method to solve sudoku board and between the methods they take up roughly the same amount of space.

    c. Maintainability

GLM 5.3 writes a new writeGrid() while DeepSeek V4.1 uses “ofstream out(solutionFilename);” which is print(ostream& out) under the hood. Personally I am a fan of **deepseek** because it’s less lines of code to accomplish the same task while being supported from all C++ versions.

3. Which of the two GenAI code examples would you select? Why?
- Based on this analysis, I would select DeepSeek V4.1 because it does the same tasks while adding less lines of code that isn’t necessary for the code to work while doing the same task.

4. How would you improve the following for the one you selected
    a. Correctness

I would ensure that they don't put in an empty board because there are over 6.67 sextillion solutions crashing my VSCode. The code also can not tell the difference between file-handling behavior so the program distinguishes between an unsolvable puzzle and a failure to read or write a file.

    b. Execution time

Because this uses a recursive solution, I don't see why you would try to optimize this because if you would like very performant code you would have not selected a recursive solution to solve sudoku puzzle.

    c. Space complexity

Although you can make this program more space efficient, it is not worth it to fix it in my eyes. The only thing you could optimize is to make sure that you are not pasting the grids multiple times.

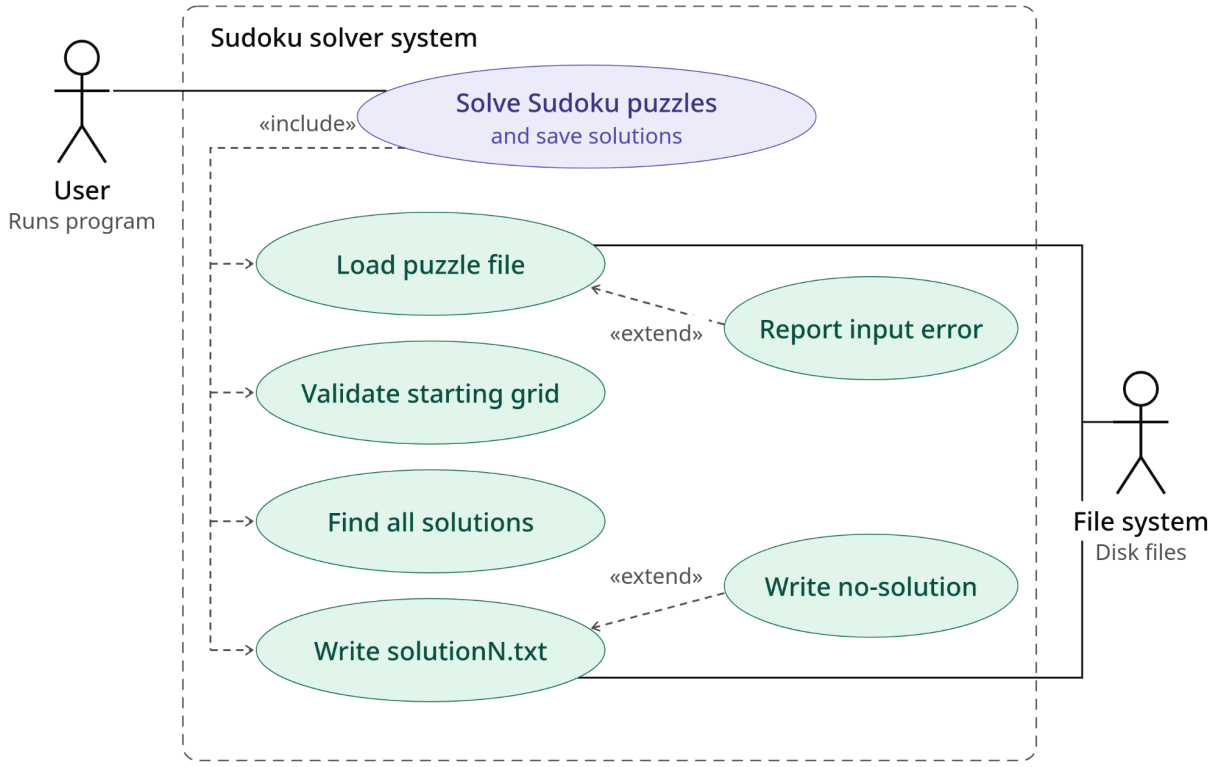
    d. Maintainability

separate the program's responsibilities into clearly defined functions like what GLM did. This will allow for more abstractions that will allow for the code to be more maintainable rather than everything being inside the main function.

The new prompt I gave to the AI was

"I would ensure that they don't put in an empty board because there are over 6.67 sextillion solutions crashing my VSCode. The code also can not tell the difference between file-handling behavior so the program distinguishes between an unsolvable puzzle and a failure to read or write a file."

The Raw Code is in the files: generated comparison



How the program works:

The user runs the program. It reads puzzle1.txt to puzzle5.txt, finds every solution to each puzzle with recursive backtracking, and writes the results to solution1.txt to solution5.txt.

Sudoku Solver - Use Case Documentation

1. Identify / Discover a Use Case

Primary use case: Solve Sudoku Puzzles and Save Solutions. Primary actor: User. Secondary actor: File System (supplies puzzleN.txt and receives solutionN.txt). System: Sudoku Solver.

Supporting use cases (shown in the use case diagram): Load Puzzle File, Validate Starting Grid, Find All Solutions, and Write solutionN.txt are included in the primary use case. Report Input Error extends Load Puzzle File, and Write No-Solution extends Write solutionN.txt.

2. Brief Description

The user runs a C++ Sudoku solver. The system reads five text files (puzzle1.txt through puzzle5.txt) containing 9x9 Sudoku grids, prints each original puzzle to the console, and searches for all valid solutions using recursive backtracking. Every solution is written to a matching output file (solution1.txt through solution5.txt) in the same format as the input files, followed by the line "Solutions found: n". If the puzzle is invalid, cannot be loaded, or has no solution, the output file contains the single line "No solution found."

3. Step-by-Step Flow of Events

1. The user starts the compiled Sudoku Solver program.
2. For each number N from 1 to 5, the system builds the file names puzzleN.txt and solutionN.txt, prints a header, and creates a new Sudoku object.
3. The system attempts to open and read puzzleN.txt, expecting exactly 81 cells containing digits 1-9 or underscores.
4. If loading fails, the system prints a specific error message (could not open file, incomplete puzzle, invalid character, or more than 81 cells), writes "No solution found." to solutionN.txt, and continues with the next puzzle.
5. For a successfully loaded file, the system displays the original Sudoku grid on the console.
6. The system opens solutionN.txt for writing. If the file cannot be opened, it prints an error and continues with the next puzzle.
7. The system checks that the existing numbers do not conflict within rows, columns, or 3x3 blocks.
8. If the starting grid is invalid, the system reports the conflict, writes "No solution found." to the console and to solutionN.txt, and continues with the next puzzle.
9. Otherwise, the system recursively chooses the empty cell with the fewest valid candidates, tries each digit, and backtracks as needed.

10. Each time a complete valid grid is reached, the system increments the solution count and writes that grid to solutionN.txt (nine rows followed by a blank line).
11. When the search ends, the system writes "Solutions found: n" to solutionN.txt and prints the same line to the console. If n is zero, it writes and prints "No solution found." instead.
12. The system closes the output file, repeats for the remaining puzzles, and then exits.

4. Detailed Use Case Description

Use case name: Solve Sudoku Puzzles and Save Solutions

Primary actor: User

Secondary actor: File System

Trigger: The user executes the compiled Sudoku Solver program.

Preconditions: The program is compiled. Files puzzle1.txt through puzzle5.txt are in the program's working directory, and the user has permission to create files in that directory.

Main success scenario: A puzzle file is read and validated, the original grid is shown on the console, recursive backtracking finds every valid completed grid, and all solutions are written to solutionN.txt followed by the total count. The console shows the count for each puzzle.

Alternate flows: A missing file, incomplete grid, invalid character, or extra data produces a specific console error message, and solutionN.txt is created containing "No solution found." Conflicting starting clues or an unsolvable grid also produce "No solution found." in the console and in solutionN.txt. A puzzle with multiple solutions writes each one to the file before the count. If the output file cannot be opened, an error is printed and that puzzle is skipped.

Postconditions: Every available puzzle has been processed and a solutionN.txt file exists for each one. No input puzzle file is modified. Solutions are streamed to the file as they are found and are not stored in memory.

Business rules: Each completed row, column, and 3x3 block contains the digits 1 through 9 exactly once. Underscores represent blank cells. Solution files use the same layout as the puzzle files: nine rows of nine space-separated cells.

Known limitation: A completely empty or nearly empty board has an astronomically large number of solutions (about 6.67 x 10^21 for an empty board), so the program would write output indefinitely. This edge case is not handled in the current version and we just error out the code saying that it’s not possible because it crashes by VScode when it tries to generate all sextillion possibilities.

GLM Code:

```
/*
 * Program: EECS 348 Assignment 4 (Modified) - Sudoku Solver
 * Description: Solves five Sudoku puzzles using object-oriented
 * programming and recursive backtracking. Instead of only printing
 * to the console, every puzzle's solutions are written to a file
 * named solutionN.txt (N = puzzle number).
 *
 * Inputs: puzzle1.txt - puzzle5.txt
 *
 * Output: solution1.txt - solution5.txt, where each file contains:
 *         - "No solution found." if the puzzle has no solutions, or
 *         - all solutions, followed by the line
 *           "Solutions found: n" (n = number of solutions).
 *         Solutions use the same format as the input files.
 *         A brief status is also printed to the console.
 *
 * Author: Ethan Church
 * Creation Date: October 13, 2026
 * Collaborators: Trey Curtis
 * Sources: ChatGPT, Gemini
 *
 * Improvements:
 * - Solutions are now saved to solutionN.txt files.
```

```
 * - Added output-file error handling.
 * - Kept the optimized recursive backtracking method.
 *
 *
 * Author: Eric Chen
 * Creation Date: October 28th, 2026
 * Sources: GLM
 *
 */

#include <iostream>
#include <fstream>
#include <string>

using namespace std;

class Sudoku {
private:
    int grid[9][9]{};
    long long solutions = 0;
    string errorMessage;

    // Stream for the solutionN.txt output file.
    ofstream outputFile;
```

```
/*
 * Checks whether a number can be placed in a cell.
 * The number cannot already exist in the same row,
 * column, or 3x3 block.
 */
bool canPlace(int row, int col, int digit) const {

    // Check the row and column.
    for (int i = 0; i < 9; i++) {
        if (grid[row][i] == digit ||
            grid[i][col] == digit) {
            return false;
        }
    }

    // Find the beginning of the 3x3 block.
    int startRow = (row / 3) * 3;
    int startCol = (col / 3) * 3;

    // Check every cell inside the block.
    for (int r = startRow; r < startRow + 3; r++) {
        for (int c = startCol; c < startCol + 3; c++) {
            if (grid[r][c] == digit) {
```

```
                return false;
            }
        }
    }

    return true;
}

/*
 * Writes the current solved grid to the solution file.
 * Uses the same format as the input puzzle files:
 * nine space-separated values per row.
 */
void writeGrid() {

    for (int row = 0; row < 9; row++) {
        for (int col = 0; col < 9; col++) {
            outputFile << grid[row][col] << " ";
        }

        outputFile << endl;
    }

    // Blank line separating solutions / count line.
```

```
    outputFile << endl;
}

/*
 * Uses recursive backtracking to solve the puzzle.
 * Chooses the empty cell with the fewest valid numbers.
 * If a guess does not work, it removes the number
 * and tries another possibility. Every completed grid
 * is written to the solution file.
 */
void backtrack() {

    int bestRow = -1;
    int bestCol = -1;
    int fewest = 10;

    // Search for the empty cell with the fewest choices.
    for (int row = 0; row < 9; row++) {
        for (int col = 0; col < 9; col++) {

            // Skip cells that already contain a number.
            if (grid[row][col] != 0) {
                continue;
            }
```

```
        int choices = 0;

        // Count the valid numbers for this cell.
        for (int digit = 1; digit <= 9; digit++) {
            if (canPlace(row, col, digit)) {
                choices++;
            }
        }

        // No valid numbers means this path cannot work.
        if (choices == 0) {
            return;
        }

        // Remember the cell with the fewest choices.
        if (choices < fewest) {
            fewest = choices;
            bestRow = row;
            bestCol = col;
        }
    }
}
```

```
/*
 * If no empty cells remain, the puzzle is solved.
 * Write the solution to the file and continue
 * searching for any additional solutions.
 */
if (bestRow == -1) {
    solutions++;

    writeGrid();

    return;
}

// Try numbers 1 through 9 in the selected cell.
for (int digit = 1; digit <= 9; digit++) {

    if (canPlace(bestRow, bestCol, digit)) {

        // Place the number.
        grid[bestRow][bestCol] = digit;

        // Recursively solve the remaining puzzle.
        backtrack();
```

```
                // Remove the number to try another possibility.
                grid[bestRow][bestCol] = 0;
            }
        }
    }

public:

    /*
     * Reads a Sudoku puzzle from a text file.
     * Underscores represent empty cells.
     * Displays a specific error if the file is invalid.
     */
    bool load(const string& filename) {

        ifstream input(filename);

        // Check whether the file exists and can be opened.
        if (!input.is_open()) {
            errorMessage = "Could not open file: " + filename;
            return false;
        }

        char cell;
```

```
// Read all 81 cells from the file.
for (int row = 0; row < 9; row++) {
    for (int col = 0; col < 9; col++) {

        // Check for missing puzzle data.
        if (!(input >> cell)) {
            errorMessage =
                "Incomplete puzzle: Expected 81 cells.";
            return false;
        }

        // Convert underscores into empty cells.
        if (cell == '_') {
            grid[row][col] = 0;
        }

        // Convert character digits into integers.
        else if (cell >= '1' && cell <= '9') {
            grid[row][col] = cell - '0';
        }

        // Reject invalid characters.
        else {
```

```
                errorMessage =
                    "Invalid character found in puzzle.";
                return false;
            }
        }
    }

    // Check whether the file contains extra data.
    if (input >> cell) {
        errorMessage =
            "Invalid puzzle: More than 81 cells found.";
        return false;
    }

    solutions = 0;
    errorMessage = "";

    return true;
}

/*
 * Prints the Sudoku grid to the console.
 * Empty cells are displayed as underscores.
 */
```

```
void print() const {

    for (int row = 0; row < 9; row++) {
        for (int col = 0; col < 9; col++) {

            if (grid[row][col] == 0) {
                cout << "_";
            }
            else {
                cout << grid[row][col];
            }

            if (col < 8) {
                cout << " ";
            }
        }

        cout << endl;
    }

    cout << endl;
}

/*
```

```
 * Checks whether the original puzzle is valid.
 * Filled cells cannot contain duplicate numbers
 * in the same row, column, or 3x3 block.
 */
bool validStartingGrid() const {

    for (int row = 0; row < 9; row++) {
        for (int col = 0; col < 9; col++) {

            int digit = grid[row][col];

            // Empty cells do not need to be checked.
            if (digit == 0) {
                continue;
            }

            // Check for duplicates in rows and columns.
            for (int i = 0; i < 9; i++) {

                if (i != col && grid[row][i] == digit) {
                    return false;
                }

                if (i != row && grid[i][col] == digit) {
```

```
                return false;
            }
        }

        // Find the starting position of the 3x3 block.
        int startRow = (row / 3) * 3;
        int startCol = (col / 3) * 3;

        // Check the block for duplicate numbers.
        for (int r = startRow; r < startRow + 3; r++) {
            for (int c = startCol; c < startCol + 3; c++) {

                if ((r != row || c != col) &&
                    grid[r][c] == digit) {
                    return false;
                }
            }
        }
    }

    return true;
}
```

```
// Returns the error message if loading fails.
string getError() const {
    return errorMessage;
}

/*
 * Starts the solving process and writes the results to
 * the given solution file.
 *
 * - Invalid puzzle or no solutions:
 *       writes the single line "No solution found."
 * - Otherwise:
 *       writes all solutions (input-file format),
 *       then the line "Solutions found: n".
 */
void solve(const string& outputFilename) {

    // Open the solution file for writing.
    outputFile.open(outputFilename);

    if (!outputFile.is_open()) {
        cout << "Error: Could not create file: "
             << outputFilename << endl;
        return;
```

```
}

// A conflicting starting grid cannot have any solution.
if (!validStartingGrid()) {
    cout << "Invalid starting puzzle: "
         << "Conflicting numbers found." << endl;

    outputFile << "No solution found." << endl;
    outputFile.close();
    return;
}

// Start recursive backtracking.
// Each completed grid is written to the file.
backtrack();

// Report if no valid solutions were found.
if (solutions == 0) {
    outputFile << "No solution found." << endl;
}

// Write the total number of solutions.
else {
    outputFile << "Solutions found: "
```

```
                << solutions << endl;
        }

        outputFile.close();
    }

    // Returns the number of solutions found.
    long long getSolutions() const {
        return solutions;
    }
};

/*
 * Main program:
 * Reads and solves all five Sudoku puzzle files.
 * Each puzzle uses a new Sudoku object, and its solutions
 * are written to solutionN.txt (N = puzzle number).
 */
int main() {

    for (int number = 1; number <= 5; number++) {

        // Create the filenames for the current puzzle.
        string filename =
```

```
    "puzzle" + to_string(number) + ".txt";

string outputFilename =
    "solution" + to_string(number) + ".txt";

// Create a new Sudoku object.
Sudoku puzzle;

cout << "========== " << filename
     << " ==========" << endl;

// Attempt to load the puzzle.
if (!puzzle.load(filename)) {

    // Display the specific error message.
    cout << "Error: " << puzzle.getError()
         << endl << endl;

    // Continue to the next puzzle.
    continue;
}

// Display the original puzzle.
cout << "Original Puzzle:" << endl;
```

```
        puzzle.print();

        // Solve the puzzle and write its solutions
        // to the solutionN.txt file.
        puzzle.solve(outputFilename);

        // Report the result in the console.
        if (puzzle.getSolutions() > 0) {
            cout << "Solutions found: "
                 << puzzle.getSolutions()
                 << " (written to "
                 << outputFilename << ")" << endl;
        }
        else {
            cout << "No solution found (written to "
                 << outputFilename << ")" << endl;
        }

        cout << endl;
    }

    return 0;
}
```

DeepSeek Code: